

Documentación Técnica

auth-service

Microservicio de autenticación y usuarios

Proyecto: **Explore Costa Rica Tours**

Tecnología principal: Java 21, Spring Boot 3.3.5
Seguridad: Spring Security 6, sesiones por token UUID
Persistencia: Spring Data JPA, Flyway
Ambientes: Local, Render PostgreSQL, Azure App Service
Versión documentada: 1.0.0

8 de junio de 2026

Índice

1. Resumen ejecutivo	3
2. Objetivo del microservicio	3
2.1. Funciones principales	3
3. Stack tecnológico	3
4. Arquitectura del proyecto	4
4.1. Estructura principal de paquetes	4
4.2. Diagrama lógico	5
5. Modelo de dominio	5
5.1. Usuario	5
5.2. Sesión	5
6. Base de datos	6
6.1. Tabla users	6
6.2. Tabla sessions	6
7. Seguridad	7
7.1. Estrategia de autenticación	7
7.2. Flujo de login	7
7.3. Validación de peticiones protegidas	7
7.4. Reglas de autorización	7
7.5. Cifrado de contraseñas	7
8. Endpoints REST	8
8.1. Resumen de endpoints	8
8.2. POST /auth/register	8
8.3. POST /auth/login	9
8.4. GET /auth/me	9
8.5. POST /auth/logout	10
8.6. GET /users	10
8.7. GET /users/{id}	11
8.8. PUT /users/{id}	11
8.9. DELETE /users/{id}	11
8.10. GET /health	11
9. Manejo de errores	12
10. Configuración por ambiente	12
10.1. application.yml	12
10.2. Perfil local	13
10.3. Perfil Render	13
10.4. Perfil Azure SQL	13

11.Variables de entorno	13
12.Ejecución local	14
13.Contenerización con Docker	14
14.CI/CD y despliegue	15
14.1. Flujo de despliegue	15
14.2. URL productiva documentada	15
15.Integración con otros microservicios	15
16.Pruebas automatizadas	16
17.Consideraciones de seguridad y mantenimiento	16
18.Conclusión	17

1 Resumen ejecutivo

El microservicio **auth-service** centraliza la autenticación y la administración de usuarios para el ecosistema de **Explore Costa Rica Tours**. Su responsabilidad principal es permitir el registro de usuarios, el inicio de sesión, la consulta del usuario autenticado, el cierre de sesión y la administración de usuarios por parte de perfiles con rol **ADMIN**.

La autenticación no utiliza JWT. En su lugar, el sistema genera un token UUID por sesión y lo almacena en la base de datos dentro de la tabla **sessions**. Cada petición protegida debe enviar el token en el encabezado HTTP **Authorization** usando el formato **Bearer <session-uuid>**.

2 Objetivo del microservicio

El objetivo de este microservicio es separar la lógica de autenticación y usuarios del resto de la plataforma. Esto permite que otros servicios puedan delegar la validación de sesión en **auth-service** mediante el endpoint **GET /auth/me**.

2.1 Funciones principales

- Registrar usuarios nuevos con rol inicial **USER**.
- Autenticar usuarios mediante correo y contraseña.
- Generar sesiones persistidas en base de datos mediante tokens UUID.
- Validar sesiones activas y no expiradas.
- Cerrar sesión eliminando el token de la base de datos.
- Administrar usuarios desde endpoints protegidos por rol **ADMIN**.
- Crear automáticamente un primer usuario administrador si se configura **ADMIN_PASSWORD**.

3 Stack tecnológico

Componente	Tecnología
Runtime	Java 21
Framework principal	Spring Boot 3.3.5
API REST	Spring Web
Seguridad	Spring Security 6
Persistencia	Spring Data JPA / Hibernate
Validaciones	Jakarta Validation
Migraciones	Flyway
Base local	H2 en memoria
Base productiva	PostgreSQL en Render

Compatibilidad adicional	SQL Server mediante driver MSSQL y perfil Azure SQL
Documentación API	SpringDoc OpenAPI / Swagger UI
Build	Maven
Contenedores	Docker multi-stage build
CI/CD	GitHub Actions hacia Azure Container Registry y Azure App Service

4 Arquitectura del proyecto

El proyecto sigue una estructura cercana a arquitectura hexagonal o arquitectura por puertos y adaptadores. La lógica de negocio se encuentra en la capa de aplicación y dominio, mientras que los controladores REST y la persistencia son adaptadores externos.

4.1 Estructura principal de paquetes

```
src/main/java/com/exploreocr/auth
|-- adapter
| |-- in/web # Controladores REST y DTOs HTTP
| '-- out/persistence # Entidades JPA, repositorios y mappers
|-- application
| |-- exception # Excepciones de negocio HTTP-aware
| '-- service # Casos de uso implementados
|-- domain
| |-- model # Modelos de dominio: User, Session, Role
| '-- port # Puertos de entrada y salida
'-- infrastructure
    |-- config # CORS y OpenAPI
    '-- security # Spring Security y filtro de sesión
```

4.2 Diagrama lógico

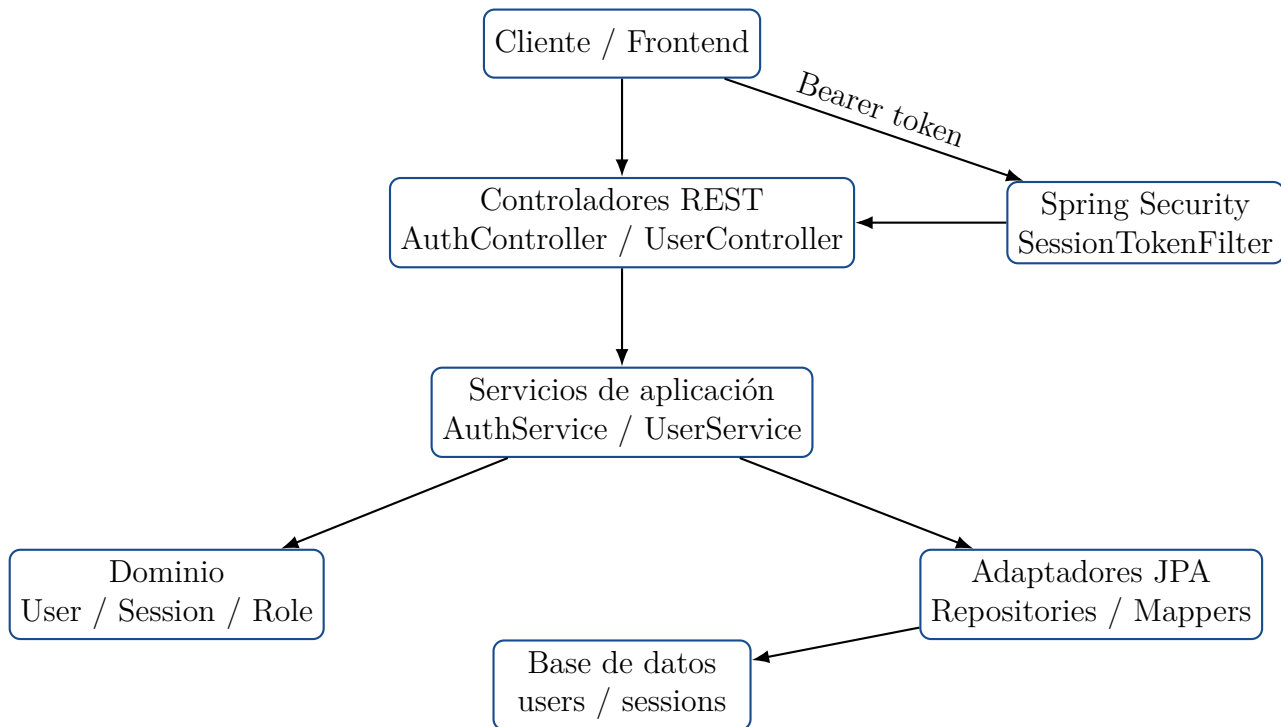


Figura 1: Arquitectura lógica del microservicio `auth-service`.

5 Modelo de dominio

5.1 Usuario

El modelo `User` representa una cuenta registrada en el sistema.

Campo	Tipo	Descripción
<code>id</code>	UUID	Identificador único del usuario.
<code>name</code>	String	Nombre visible del usuario.
<code>email</code>	String	Correo único, normalizado en minúsculas.
<code>passwordHash</code>	String	Contraseña cifrada con BCrypt.
<code>role</code>	Role	Rol del usuario: <code>USER</code> o <code>ADMIN</code> .
<code>avatarUrl</code>	String	URL opcional de avatar.
<code>createdAt</code>	LocalDateTime	Fecha de creación.
<code>updatedAt</code>	LocalDateTime	Fecha de última actualización.

5.2 Sesión

El modelo `Session` representa una sesión activa almacenada en base de datos.

Campo	Tipo	Descripción
id	UUID	Identificador único de la sesión.
user	User	Usuario dueño de la sesión.
token	String	Token UUID enviado al cliente.
expiresAt	LocalDateTime	Fecha y hora de expiración.
createdAt	LocalDateTime	Fecha de creación de la sesión.

La sesión se considera expirada cuando **expiresAt** no es posterior a la fecha y hora actual.

6 Base de datos

6.1 Tabla users

La tabla **users** almacena los datos de las cuentas del sistema.

```
CREATE TABLE users (  
  id UUID DEFAULT gen_random_uuid() PRIMARY KEY,  
  name VARCHAR(100) NOT NULL,  
  email VARCHAR(255) NOT NULL UNIQUE,  
  password_hash VARCHAR(255) NOT NULL,  
  role VARCHAR(20) NOT NULL DEFAULT 'USER' CHECK (role IN ('USER', 'ADMIN')),  
  avatar_url VARCHAR(500),  
  created_at TIMESTAMP DEFAULT NOW() NOT NULL,  
  updated_at TIMESTAMP DEFAULT NOW() NOT NULL  
);  
  
CREATE INDEX idx_users_email ON users(email);
```

6.2 Tabla sessions

La tabla **sessions** almacena los tokens activos. Cada sesión pertenece a un usuario. Si el usuario se elimina, sus sesiones se eliminan en cascada.

```
CREATE TABLE sessions (  
  id UUID DEFAULT gen_random_uuid() PRIMARY KEY,  
  user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,  
  token VARCHAR(255) NOT NULL UNIQUE,  
  expires_at TIMESTAMP NOT NULL,  
  created_at TIMESTAMP DEFAULT NOW() NOT NULL  
);  
  
CREATE INDEX idx_sessions_token ON sessions(token);  
CREATE INDEX idx_sessions_user_id ON sessions(user_id);
```

7 Seguridad

7.1 Estrategia de autenticación

El servicio usa autenticación stateless desde la perspectiva de Spring Security, pero la sesión se persiste en base de datos. El token no contiene información firmada como en JWT; es un UUID opaco que debe buscarse en la tabla `sessions`.

7.2 Flujo de login

1. El cliente envía correo y contraseña a `POST /auth/login`.
2. El servicio busca el usuario por correo ignorando mayúsculas y minúsculas.
3. La contraseña recibida se compara contra `password_hash` usando BCrypt.
4. Si las credenciales son correctas, se genera un token UUID.
5. El token se guarda en `sessions` con una fecha de expiración.
6. El cliente recibe el token y los datos públicos del usuario.

7.3 Validación de peticiones protegidas

El filtro `SessionTokenFilter` intercepta las peticiones HTTP. Si encuentra un encabezado `Authorization` con formato `Bearer <token>`, busca la sesión por token y valida que no esté expirada. Si la sesión es válida, coloca el usuario autenticado en el contexto de seguridad de Spring.

7.4 Reglas de autorización

Ruta	Regla
<code>/auth/login</code>	Público.
<code>/auth/register</code>	Público.
<code>/swagger-ui/**,</code> <code>/api-docs/**,</code> <code>/v3/api-docs/**</code>	Público.
<code>/health,</code> <code>/h2-console/**</code>	<code>/error,</code> Público.
<code>/users/**</code>	Requiere rol ADMIN.
Cualquier otra ruta	Requiere usuario autenticado.

7.5 Cifrado de contraseñas

Las contraseñas se almacenan con BCrypt usando fuerza 12. El sistema nunca retorna el hash de contraseña en respuestas HTTP.

8 Endpoints REST

8.1 Resumen de endpoints

Método	Endpoint	Acceso	Descripción
POST	/auth/register	Público	Registra un usuario y crea sesión.
POST	/auth/login	Público	Autentica un usuario y crea sesión.
GET	/auth/me	Autenticado	Retorna el usuario de la sesión actual.
POST	/auth/logout	Autenticado	Elimina la sesión actual.
GET	/users	ADMIN	Lista usuarios paginados.
GET	/users/{id}	ADMIN	Consulta un usuario por ID.
PUT	/users/{id}	ADMIN	Actualiza datos de usuario.
DELETE	/users/{id}	ADMIN	Elimina un usuario.
GET	/health	Público	Verifica salud del servicio.

8.2 POST /auth/register

Registra un usuario nuevo con rol **USER**. Valida que el correo no exista y que **password** y **confirmPassword** coincidan.

Request:

```
{
  "name": "Juan Costa",
  "email": "juan@explorecr.com",
  "password": "MiPassword123!",
  "confirmPassword": "MiPassword123!"
}
```

Response 201:

```
{
  "data": {
    "token": "session-uuid",
    "user": {
      "id": "user-uuid",
      "name": "Juan Costa",
      "email": "juan@explorecr.com",
      "role": "user",
      "avatar": null,
      "createdAt": "2026-05-01T10:00:00"
    }
  }
}
```

```
}
```

8.3 POST /auth/login

Autentica un usuario existente usando correo y contraseña.

Request:

```
{
  "email": "juan@explorecr.com",
  "password": "MiPassword123!"
}
```

Response 200:

```
{
  "data": {
    "token": "session-uuid",
    "user": {
      "id": "user-uuid",
      "name": "Juan Costa",
      "email": "juan@explorecr.com",
      "role": "user",
      "avatar": null,
      "createdAt": "2026-05-01T10:00:00"
    }
  }
}
```

8.4 GET /auth/me

Retorna el usuario autenticado a partir del token enviado.

Header requerido:

```
Authorization: Bearer <session-uuid>
```

Response 200:

```
{
  "data": {
    "id": "user-uuid",
    "name": "Juan Costa",
    "email": "juan@explorecr.com",
    "role": "user",
    "avatar": null,
    "createdAt": "2026-05-01T10:00:00"
  }
}
```

8.5 POST /auth/logout

Elimina de la base de datos la sesión asociada al token enviado.

Header requerido:

```
Authorization: Bearer <session-uuid>
```

Response 200:

```
{
  "message": "Logged out successfully"
}
```

8.6 GET /users

Lista usuarios de forma paginada. Este endpoint requiere rol ADMIN.

Parámetros de consulta:

Parámetro	Default	Descripción
page	1	Número de página. El servicio asegura un mínimo de 1.
pageSize	10	Tamaño de página. El servicio limita el máximo a 100.
search	null	Busca por nombre o correo.
role	null	Filtra por rol: USER o ADMIN.

Ejemplo:

```
GET /users?page=1&pageSize=10&search=juan&role=user
Authorization: Bearer <admin-session-uuid>
```

Response 200:

```
{
  "data": [
    {
      "id": "user-uuid",
      "name": "Juan Costa",
      "email": "juan@explorecr.com",
      "role": "user",
      "avatar": null,
      "createdAt": "2026-05-01T10:00:00"
    }
  ],
  "total": 1,
  "page": 1,
  "pageSize": 10,
  "totalPages": 1
}
```

8.7 GET /users/{id}

Consulta un usuario específico por UUID. Requiere rol ADMIN.

```
GET /users/<user-uuid>
Authorization: Bearer <admin-session-uuid>
```

8.8 PUT /users/{id}

Actualiza los datos editables de un usuario. Requiere rol ADMIN. Si algún campo viene en null, se conserva el valor actual.

Request:

```
{
  "name": "Juan Actualizado",
  "email": "juan.actualizado@explorecr.com",
  "role": "admin",
  "avatar": "https://example.com/avatar.png"
}
```

Response 200:

```
{
  "data": {
    "id": "user-uuid",
    "name": "Juan Actualizado",
    "email": "juan.actualizado@explorecr.com",
    "role": "admin",
    "avatar": "https://example.com/avatar.png",
    "createdAt": "2026-05-01T10:00:00"
  },
  "message": "User updated successfully"
}
```

8.9 DELETE /users/{id}

Elimina un usuario. Requiere rol ADMIN. El servicio impide que el usuario autenticado elimine su propia cuenta. Antes de eliminar el usuario, borra sus sesiones asociadas.

Response 200:

```
{
  "message": "User deleted successfully"
}
```

8.10 GET /health

Endpoint público para revisar que el servicio esté activo.

Response 200:

```
{
  "status": "ok",
  "service": "auth-service"
}
```

9 Manejo de errores

El servicio centraliza el manejo de errores mediante `GlobalExceptionHandler`. Las excepciones de negocio extienden de `ApiException`, que incluye un mensaje y un código HTTP.

Excepción / caso	HTTP	Mensaje
<code>InvalidCredentialsException</code>	401	Invalid email or password
<code>PasswordMismatchException</code>	400	Passwords do not match
<code>UserAlreadyExistsException</code>	409	Email already in use
<code>UserNotFoundException</code>	404	User not found
<code>SelfDeleteException</code>	403	Cannot delete your own account
Token inválido o expirado	401	Invalid or expired session
Acceso sin permiso	403	Access denied
Validación de request inválida	400	Invalid request

Formato de error:

```
{
  "error": "Invalid email or password",
  "status": 401
}
```

10 Configuración por ambiente

10.1 application.yml

La configuración base define nombre de aplicación, puerto, Swagger, Flyway, duración de sesión, CORS y credenciales iniciales del administrador.

```
server:
  port: ${PORT:8080}

app:
  session:
    expiration-hours: ${SESSION_EXPIRATION_HOURS:24}
  cors:
    allowed-origins: ${FRONTEND_URL:http://localhost:5173}
  admin:
    email: ${ADMIN_EMAIL:admin@explorecr.com}
```

```
password: ${ADMIN_PASSWORD:}
```

10.2 Perfil local

El perfil `local` usa H2 en memoria, desactiva Flyway y permite la consola H2 en `/h2-console`.

```
spring:
  datasource:
    url: jdbc:h2:mem:authdb;DB_CLOSE_DELAY=-1;MODE=PostgreSQL
  jpa:
    hibernate:
      ddl-auto: create-drop
  flyway:
    enabled: false
```

10.3 Perfil Render

El perfil `render` usa PostgreSQL y valida el esquema mediante Hibernate. Las migraciones Flyway se mantienen activas.

```
spring:
  datasource:
    url: ${RENDER_POSTGRES_URL}
    username: ${RENDER_POSTGRES_USER}
    password: ${RENDER_POSTGRES_PASSWORD}
  jpa:
    hibernate:
      ddl-auto: validate
  flyway:
    enabled: true
```

10.4 Perfil Azure SQL

El proyecto incluye configuración para SQL Server mediante `application-azure.yml`. Sin embargo, las migraciones SQL actuales usan `pgcrypto` y `gen_random_uuid()`, por lo que para Azure SQL se requerirían ajustes de migración y dialecto si se decide usar SQL Server como base principal.

11 Variables de entorno

Variable	Uso
SPRING_PROFILES_ACTIVE	Perfil activo para ejecución local.
SPRING_PROFILE	Perfil usado por el contenedor Docker al arrancar. Por defecto: <code>render</code> .

PORT	Puerto HTTP de la aplicación. Por defecto: 8080.
FRONTEND_URL	Origen permitido para CORS. Puede contener varios orígenes separados por coma.
SESSION_EXPIRATION_HOURS	Duración de la sesión en horas. Por defecto: 24.
ADMIN_EMAIL	Correo del administrador inicial.
ADMIN_PASSWORD	Contraseña del administrador inicial. Si está vacía, no se crea admin.
RENDER_POSTGRES_URL	JDBC URL de PostgreSQL en Render.
RENDER_POSTGRES_USER	Usuario de PostgreSQL.
RENDER_POSTGRES_PASSWORD	Contraseña de PostgreSQL.
AZURE_SQL_URL	JDBC URL para SQL Server si se usa perfil Azure SQL.
AZURE_SQL_USERNAME	Usuario SQL Server.
AZURE_SQL_PASSWORD	Contraseña SQL Server.

12 Ejecución local

Para ejecutar localmente con H2:

```
mvn spring-boot:run -Dspring-boot.run.profiles=local
```

Servicio local:

```
http://localhost:8080
```

Swagger UI:

```
http://localhost:8080/swagger-ui/index.html
```

Health check:

```
http://localhost:8080/health
```

13 Contenerización con Docker

El Dockerfile usa un build multi-stage. Primero compila el JAR con Maven usando Java 21 y luego ejecuta el artefacto en una imagen liviana con JRE 21.

```
FROM maven:3.9-eclipse-temurin-21-alpine AS build
WORKDIR /workspace
COPY pom.xml .
COPY src ./src
RUN mvn clean package -DskipTests

FROM eclipse-temurin:21-jre-alpine
WORKDIR /app
COPY --from=build /workspace/target/auth-service-*.jar app.jar
EXPOSE 8080
ENTRYPOINT ["sh", "-c", "java -Dspring.profiles.active=${SPRING_PROFILE:-render} -jar app.jar"]
```

14 CI/CD y despliegue

El repositorio incluye el workflow `deploy-auth-service.yml`. Este se ejecuta al hacer push a la rama `main` cuando cambian archivos del código fuente, el `pom.xml`, el `Dockerfile` o el propio workflow.

14.1 Flujo de despliegue

1. Se descarga el código con `actions/checkout@v4`.
2. Se configura Java 21 con `actions/setup-java@v4`.
3. Se ejecuta `mvn clean package`.
4. Se construye la imagen Docker con etiqueta basada en el SHA del commit.
5. Se etiqueta también como `latest`.
6. Se inicia sesión en Azure Container Registry usando secretos de GitHub.
7. Se publica la imagen en ACR.
8. Se despliega la imagen en Azure App Service con `azure/webapps-deploy@v3`.

14.2 URL productiva documentada

```
https://explore-cr-auth-service-d3gzhsdjazexh9er.eastus2-01.azurewebsites.net
```

Swagger en producción:

```
https://explore-cr-auth-service-d3gzhsdjazexh9er.eastus2-01.azurewebsites.net/swagger-ui/index.html
```

Health check en producción:

```
https://explore-cr-auth-service-d3gzhsdjazexh9er.eastus2-01.azurewebsites.net/health
```

15 Integración con otros microservicios

Los demás microservicios pueden validar la sesión de un usuario llamando al endpoint `GET /auth/me`. Este endpoint retorna los datos públicos del usuario si el token es válido y no ha expirado.

```
GET /auth/me HTTP/1.1
Host: explore-cr-auth-service-d3gzhsdjazexh9er.eastus2-01.azurewebsites.net
Authorization: Bearer <session-uuid>
```

Respuesta válida:


```
{
  "data": {
    "id": "user-uuid",
    "name": "Admin",
    "email": "admin@explorecr.com",
    "role": "admin",
    "avatar": null,
    "createdAt": "2026-05-01T10:00:00"
  }
}
```

Si el token no existe, está expirado o no se envía correctamente, el servicio responde con 401 Unauthorized.

16 Pruebas automatizadas

El proyecto incluye pruebas de flujo de autenticación usando **MockMvc** con perfil **local**. Las pruebas verifican dos comportamientos principales:

- Un usuario puede registrarse, recibir token y consultar `/auth/me`.
- Un administrador inicial puede iniciar sesión y listar usuarios desde `/users`.

Comando de pruebas:

```
mvn test
```

17 Consideraciones de seguridad y mantenimiento

- Mantener `ADMIN_PASSWORD`, credenciales de base de datos y secretos de Azure fuera del repositorio.
- Configurar `FRONTEND_URL` con los dominios reales permitidos en producción.
- Rotar credenciales de administrador si fueron compartidas durante pruebas.
- Revisar periódicamente la tabla `sessions` para limpiar sesiones expiradas. El puerto de repositorio ya contempla `deleteExpiredBefore`, aunque no se observa un scheduler activo en el código documentado.
- Evaluar si se requiere invalidar sesiones anteriores al iniciar sesión desde un nuevo dispositivo.
- Considerar rate limiting para endpoints de login y registro en ambientes públicos.
- Validar que HTTPS esté activo en Azure App Service para proteger el token de sesión durante el transporte.

18 Conclusión

El microservicio **auth-service** implementa una base sólida para autenticación centralizada dentro de Explore Costa Rica Tours. Su diseño separa controladores, casos de uso, dominio y persistencia, lo que facilita mantenimiento y evolución. La estrategia de tokens UUID persistidos simplifica la invalidación de sesiones, ya que cerrar sesión equivale a eliminar el token de la base de datos. Además, la integración con Swagger, Docker, Flyway y GitHub Actions permite operar el servicio tanto en desarrollo local como en ambientes productivos.